

ECE220 Midterm 1 Review

Author: Members of HKN

1 Memory-Mapped I/O

(a) What is the difference between a callee-saved and a caller-saved register?

(b) What is the difference between polling I/O vs. Interrupt-Driven I/O?

(c) Fill in the blanks of the following statements relating to Memory-Mapped I/O.

Certain device registers are _____ to certain _____ locations. However, the registers physically are _____ from the memory. Memory-mapped device registers are a common way to _____ computer systems with devices.

2 Traps and Subroutines

(a) Why would we need to have service routines (known as TRAPs)? Name three reasons.

(b) **Shifting** :(

Write a subroutine that performs a logical left shift of R1 by the value stored in R2, and reports the result in R1. You may assume that R2 contains a positive number. Only R6 and R1 can be modified/read from, and only R2 can be read from.

```
.ORIG x3100
SHIFTL
; code starts here
```

```
; code ends here
RET
.END
```

(c) **Permute Quarters:**

A value stored in a 16-bit LC3 register can be divided into four equal parts of four bits each:

$$X = X_1 \ X_2 \ X_3 \ X_4$$

Write a subroutine, PERMUTE, that reorders R5 as follows:

$$R5 = X_1 \ X_3 \ X_2 \ X_4$$

Assume all registers are caller-saved. You should not use loops. You may assume the existence of a DIV subroutine, which divides as expected. There are also some labels for you to use. [Hint: the previous subroutine might be useful]

```
.ORIG x3100
; assume code for entry point is omitted
;
;
;
```

```
PERMUTE
; write the code here
```

```
; code ends here
RET
; code omitted
;
.HALT ; labels below
CLEARX2X3 .FILL #0xF00F
FOUR .FILL #4
X2 .FILL 0x0F00
X3 .FILL 0x00F0
SAVER7 .FILL #0
.END
```

(d) **LC-3 Debugging**

A simple embedded temperature-monitoring system reads a sensor value and calibration data from memory before checking whether the temperature exceeds a safe operating limit. The program is intended to perform the following tasks:

- (a) Read the raw temperature sensor value stored at memory address **x4000**.
- (b) Read the calibration offset stored at memory address **x4001** and add it to the raw sensor value.
- (c) Compare the calibrated temperature against a maximum safe temperature limit of 75.
- (d) If the calibrated temperature is greater than or equal to 75, store 1 at memory address **x4002** to indicate an over-temperature condition. Otherwise, store 0.
- (e) Halt after storing the alarm status.

The program below contains **three bugs**. For each bug:

- (a) Identify the line containing the bug.
- (b) Explain what is wrong.
- (c) Describe how you would modify the program to fix it.

You may assume that the memory locations **x4000** and **x4001** have been initialized before the program starts.

```
x3000          LDI R1, SENSOR_PTR ; load the raw sensor value
x3001          JSR CALIBRATE ; apply the calibration offset
x3002          LD R3, TEMP_LIMIT ; load the maximum safe temperature
x3003          NOT R4, R3
x3004          ADD R4, R4, #1 ; form the negative of the temperature limit
x3005          ADD R4, R1, R4 ; compute calibrated temperature minus the limit
x3006          BRzp OVERHEAT ; branch if temperature is at or above the limit

x3007          AND R0, R0, #0
x3008          STI R0, ALARM ; store 0 for a safe temperature
x3009          BRnzp DONE

x300A  OVERHEAT  AND R0, R0, #0
x300B          ADD R0, R0, #1
x300C          STI R0, ALARM ; store 1 for an over-temperature condition

x300D  DONE     HALT

x300E  SENSOR_PTR .FILL x4000
x300F  CAL_OFFSET_PTR .FILL x4001
x3010  ALARM      .FILL x4002
x3011  SAVER7     .BLKW 1

x3012  CALIBRATE  JSR ADD_OFFSET ; add the calibration offset
x3013          ST R7, SAVER7 ; save R7
x3014          ADD R0, R1, #0 ; copy the calibrated value to R0
x3015          LD R7, SAVER7 ; restore R7
x3016          RET

x3017  ADD_OFFSET LD R2, CAL_OFFSET_PTR ; load the calibration offset
x3018          ADD R1, R1, R2
x3019          RET
```

```

x301A          .BLKW 300
x3146  TEMP_LIMIT  .FILL #75

```

BUG 1

Line:
Issue:
Correction:

BUG 2

Line:
Issue:
Correction:

BUG 3

Line:
Issue:
Correction:

(e) Trap Concepts

Given the figures below, determine the memory address of the TRAP vector table that will be accessed and which TRAP service routine will be executed.

User Program ASM Code	
	;
	;
TRAP 0xAA	
	;
	;

Address	Value
0xAA	0x05C0
0x1AA	0x05D0
0x2AA	0x05E0
0x3AA	0x05F0
...	
0x05C0	Routine α
0x05D0	Routine ω
0x35C0	Routine κ
0x35D0	Routine π

Trap Vector Table Entry:

- 0xAA
- 0x1AA
- 0x2AA
- 0x3AA

Trap Routine Executed:

- α

- b. ω
- c. κ
- d. π

Of the following steps executed during a TRAP, in what order are they executed?

- a. Return to User Program
- b. Execute Trap Routine
- c. Access Trap Vector Table

- 1)
- 2)
- 3)

3 Stack Operations

(a) Given the following input sequence of numbers: “24609846117”, write a sequence of pushes and pops that produces this output: “64098116472”.

(b) Two Parts:

- i. Write the expression $((4 * 2) + 1) / 3 + 5$ in postfix notation.
- ii. Write the following postfix expressions in mathematical notation and indicate what they evaluate to (if they are not valid, write “not valid”)

* 6721*-5/*

* 83+632-+1*+-

* 2233*2+5-+8/-

* 89+6-44

(c) **MP2 Postfix Calculator:** This sequence is input to the console: 445+3/8*-= Draw the stack (and where the stack pointer points to) after:

- i. 5 has been input
- ii. + has been input
- iii. * has been input
- iv. = has been input

Assume that the stack pointer points to the address **one above the most recent-pushed entry**. Remember that a POP does NOT remove an item from memory but simply changes the stack pointer!

4 C Programming

(a) What will be the output of the following C Program?

```
1 int main() {
2     int i;
3     for (i = 3; i < 13; i ++ )
4     {
5         if (i % 3 == 1)
6         {
7             printf("Bong\n");
8         }
9         if (i % 2 == 0)
10        {
11            printf("Ding\n");
12            continue;
13        }
14        printf("Odd\n");
15    }
16    return 0;
17 }
18
19
```

(b) What is the return value of this program?

```
1 int foo(int x, int y);
2
3 int main()
4 {
5     int x = 3;
6     int y = 4;
7
8     x = y + foo(x,y);
9     y = x - foo(x,y);
10
11     return x + y;
12
13
14
15 }
16
17 int foo(int x, int y)
18 {
19     int a = x + y;
20     int b = x - y;
21     a += x--;
22     y ++;
23     a += (y + 1);
24     return b + a + --x;
25
26
27 }
28
29
```

5 Conceptual Questions

(a) What is the order of access for a stack abstract data type?

(b) Define overflow and underflow.

(c) True or False. Please explain your answer.

- i. Interrupts are more efficient than polling.
- ii. There are up to 8 possible TRAP service routines.
- iii. TRAPs shield programmers from system specific details.
- iv. PSR and PC are pushed to the User stack before executing an interrupt service routine.
- v. An item is deleted after being pushed off the stack.
- vi. TRAP service routines are provided as part of the system code.